

Day 20: Unique Cities Using Set

1. Day Number

Day 20

2. Topic Name

Sets

3. Connection

Previously, you stored user and payee details using lists and dictionaries.

Today, you will learn how to find **unique city names** by removing duplicate values with a set.

4. Important Topics

- Set
 - Uniqueness
 - Duplicate removal
 - `add()`
 - Converting a list into a set
-

5. Foundational Notes

What is a set?

A **set** is a Python collection that stores only unique values.

```
cities = {"Delhi", "Mumbai", "Bengaluru"}
```

Sets do not keep duplicates

```
cities = {"Delhi", "Mumbai", "Delhi"}
```

Python stores "Delhi" only once.

Conceptually, the set becomes:

```
Delhi, Mumbai
```

Creating an empty set

Use `set()`:

```
cities = set()
```

Do not use `{}` for an empty set because `{}` creates an empty dictionary.

Adding a value

Use the `add()` method:

```
cities.add("Delhi")
```

Adding the same value again does not create a duplicate:

```
cities.add("Delhi")  
cities.add("Delhi")
```

The set still contains only one "Delhi".

Converting a list to a set

A list containing duplicate values can be converted into a set:

```
unique_values = set(original_list)
```

Important note about order

Sets are **unordered collections**. Their values may not print in the same order as the original list.

6. Easy Example

Suppose you have duplicate fruit names:

```
fruits = ["Apple", "Banana", "Apple", "Mango"]  
unique_fruits = set(fruits)  
  
print(unique_fruits)
```

Possible output:

```
{'Apple', 'Banana', 'Mango'}
```

The exact order may be different because sets do not guarantee normal list order.

7. Problem Statement

Create a list of city names containing some duplicate cities.

Use a set to remove the duplicates and print only the unique city names.

Example city list:

```
Delhi, Mumbai, Delhi, Bengaluru, Mumbai
```

The result should contain:

```
Delhi, Mumbai, Bengaluru
```

Each city should appear only once.

8. Concepts Used

- List creation
 - Duplicate values
 - Set creation
 - List-to-set conversion
 - Uniqueness
 - Function definition
 - Function parameter
 - Return value
 - Empty-list checking
-

9. Thought Process

Step 1: Create the city list

Store several city names inside a list. Some names can appear more than once.

Step 2: Send the list to a function

Create a function that accepts the city list as a parameter.

Step 3: Check whether the list is empty

If the list has no cities, return an empty set or display an appropriate message.

Step 4: Create a set

Convert the city list into a set.

Because sets store only unique values, duplicate city names will automatically be removed.

Step 5: Return the set

The function should return the set of unique cities.

Step 6: Print the result

Call the function and print the returned unique cities.

10. Pseudocode

```
START

DEFINE a function that accepts city_list

    IF city_list is empty
        RETURN an empty set

    CONVERT city_list into a set
    STORE it in unique_cities

    RETURN unique_cities

CREATE a list containing duplicate city names

CALL the function with the city list
STORE the returned result

PRINT the unique cities

END
```

11. Suggested Solving Approach: Functional Approach

Use one function whose responsibility is to find unique cities.

Suggested structure:

```
Function:
    Input  → list of city names
    Work   → convert the list into a set
    Output → set of unique cities
```

Possible function name:

```
get_unique_cities()
```

The function should:

1. Accept a city list.
2. Handle an empty list.
3. Convert the list into a set.
4. Return the set.

Keep the list creation and printing outside the function.

12. Easy Edge Cases

Edge Case 1: All cities are the same

Input:

```
["Delhi", "Delhi", "Delhi"]
```

Expected result:

```
{"Delhi"}
```

Edge Case 2: Empty list

Input:

```
[]
```

Expected result:

```
No cities found
```

Or the function may return:

```
set()
```

Edge Case 3: Uppercase and lowercase names

Input:

```
["Delhi", "delhi"]
```

A set considers these different values because Python strings are case-sensitive.

Result:

```
{"Delhi", "delhi"}
```

Handling uppercase and lowercase differences is optional for today's practice.

13. Expected Input

The city names can be stored directly in a list:

```
["Delhi", "Mumbai", "Delhi", "Bengaluru", "Mumbai"]
```

No user input is required unless you want additional practice.

14. Expected Output

Possible output:

```
Unique cities:  
{'Delhi', 'Mumbai', 'Bengaluru'}
```

The order may vary, for example:

```
Unique cities:  
{'Bengaluru', 'Delhi', 'Mumbai'}
```

Both outputs are correct because sets are unordered.

15. Hint Only

Create a function that receives the city list. Inside the function, check whether the list is empty and then use:

```
set(city_list)
```

Return the result and print it after calling the function.